

 INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC		
Nombres: • Anthony Alarcon • Erick Basurto	Periodo Académico: 2025-2026	Carrera: Desarrollo de Software
Asignatura: Devops	Fecha: 04-06-2026	Código: 03
Tema: Trabajo en grupo		

Repositorio GitHub:

https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo

1. INVESTIGACIÓN

- **Control de versiones**

Sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo

Utilidad: Sirve para trabajar de forma organizada en proyectos

Ejemplo práctico:

Un desarrollador crea una página web y va guardando versiones cada vez que hace cambios.

Fuente bibliográfica consultada: Chacon, S., & Straub, B. (2014). Pro Git Book. <https://git-scm.com/book/en/v2>

- **Git.**

Sistema de control de versiones distribuido que permite registrar cambios en archivos y colaborar en proyectos.

Utilidad: Sirve para llevar historial de cambios, trabajar en equipo y recuperar versiones anteriores.

Ejemplo práctico:

Un programador guarda versiones de su código mientras desarrolla una aplicación.

Fuente bibliográfica consultada:

- **GitHub**

Plataforma en la nube que aloja repositorios Git.

Utilidad: Permite colaborar en proyectos, revisar código y gestionar equipos.

Ejemplo práctico:

Un equipo de desarrollo sube su proyecto web para trabajar juntos.

Fuente bibliográfica consultada: GitHub Docs. <https://docs.github.com>

- **Repositorio local.**

Repositorio almacenado en la computadora del desarrollador.

Utilidad: Permite trabajar sin internet y guardar cambios localmente.

Ejemplo práctico.

Un programador hace commits en su laptop antes de subirlos a GitHub.

Fuente bibliográfica consultada: Git Documentation. <https://git-scm.com/docs>

- **Repositorio remoto.**

Repositorio alojado en un servidor

Utilidad:

Permite compartir el proyecto con otros desarrolladores.

Ejemplo práctico:

Un proyecto subido a GitHub para que un equipo lo descargue.

Fuente bibliográfica consultada: GitHub Docs. <https://docs.github.com>

- **Commit.**

Registro de cambios realizados en el proyecto.

Utilidad:

Permite guardar avances con historial.

Ejemplo práctico:

“Se agregó un nuevo login” como mensaje de commit.

Fuente bibliográfica consultada: Pro Git Book. <https://git-scm.com/book/en/v2>

- **Branch (rama).**

Línea de desarrollo paralela dentro del proyecto.

Utilidad: Permite trabajar en nuevas funciones sin afectar el código principal.

Ejemplo práctico.

Una rama para desarrollar

Fuente bibliográfica consultada: Git Docs. <https://git-scm.com/docs/git-branch>

- **Merge.**

Proceso de unir ramas diferentes.

Utilidad: Integra cambios al proyecto principal.

Ejemplo práctico.

Unir la rama “login” con la rama principal (main).

Fuente bibliográfica consultada: Pro Git Book. <https://git-scm.com/book/en/v2>

- **Pull Request.**

Solicitud para integrar cambios en un repositorio.

Utilidad: Permite revisión de código antes de aceptar cambios.

Ejemplo práctico:

Un desarrollador propone su código y otro lo revisa antes de aprobarlo.

Fuente bibliográfica consultada: GitHub Guides.
<https://docs.github.com/en/get-started>

- **Fork.**

Copia de un repositorio en tu propia cuenta.

Utilidad: Permite modificar proyectos sin afectar el original.

Ejemplo práctico:

Copiar un proyecto open source para mejorarlo.

Fuente bibliográfica consultada: GitHub About. <https://github.com/about>

- **Conflictos de integración.**

Errores que ocurren cuando dos cambios chocan en el mismo archivo.

Utilidad: Obliga a decidir qué versión del código se mantiene.

Ejemplo práctico:

Dos personas editan la misma línea de código diferente.

Fuente bibliográfica consultada: Git Documentation – Merge Conflicts.
<https://git-scm.com/docs/git-merge>

- **Integración Continua (CI)**

Práctica donde los cambios de código se integran y prueban automáticamente.

Utilidad: Detecta errores rápidamente.

Ejemplo práctico:

Cada vez que alguien sube código, se ejecutan pruebas automáticas.

Fuente bibliográfica consultada: Red Hat.
<https://www.redhat.com/en/topics/devops/what-is-ci-cd>

- **Entrega Continua (Continuous Delivery).**

El software está siempre listo para ser desplegado.

Utilidad: Permite lanzar versiones rápidamente.

Ejemplo práctico:

Una app lista para publicarse con un solo clic.

Fuente bibliográfica consultada: Atlassian CD.
<https://www.atlassian.com/continuous-delivery>

- **Despliegue Continuo (Continuous Deployment)**

Cada cambio aprobado se publica automáticamente.

Utilidad: Elimina intervención manual.

Ejemplo práctico:

Cada commit exitoso se sube directamente a producción.

Fuente bibliográfica consultada: Red Hat CI/CD.

<https://www.redhat.com/en/topics/devops/what-is-ci-cd>

- **GitHub Actions**

Herramienta de GitHub para automatizar tareas (CI/CD).

Utilidad: Ejecuta pruebas, despliegues y scripts automáticamente

Ejemplo práctico:

Al hacer push, se ejecutan tests y despliegue automático.

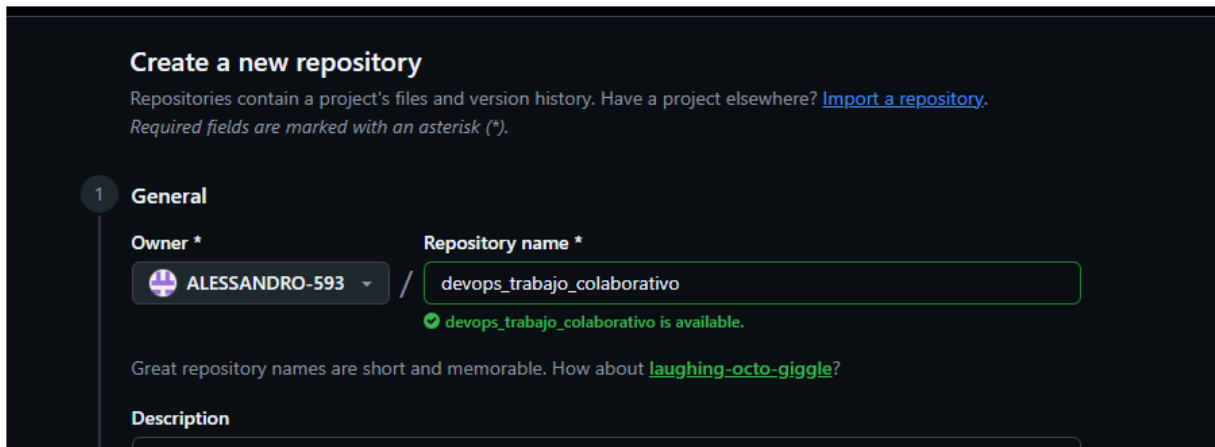
Fuente bibliográfica consultada: GitHub Actions Overview.

<https://github.com/features/actions>

2. DEMOSTRACIÓN PRÁCTICA

2.1 [Erick Basurto] Creación del repositorio en GitHub

Se da inicio al proyecto con la creación de un repositorio remoto en GitHub de visibilidad pública bajo el nombre **devops_trabajo_colaborativo**. Durante la inicialización se incluyó por defecto un archivo README.md.



The screenshot shows the 'Create a new repository' page on GitHub. The page has a dark theme. At the top, it says 'Create a new repository' and provides a link to 'Import a repository'. Below this, there's a section titled '1 General'. The 'Owner' field is set to 'ALESSANDRO-593'. The 'Repository name' field is set to 'devops_trabajo_colaborativo', and a green checkmark indicates it is available. There's a suggestion for 'laughing-octo-giggle?'. The 'Description' field is empty.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#)
Required fields are marked with an asterisk ().*

1 General

Owner *  ALESSANDRO-593 / **Repository name ***

✔ devops_trabajo_colaborativo is available.

Great repository names are short and memorable. How about [laughing-octo-giggle?](#)

Description

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

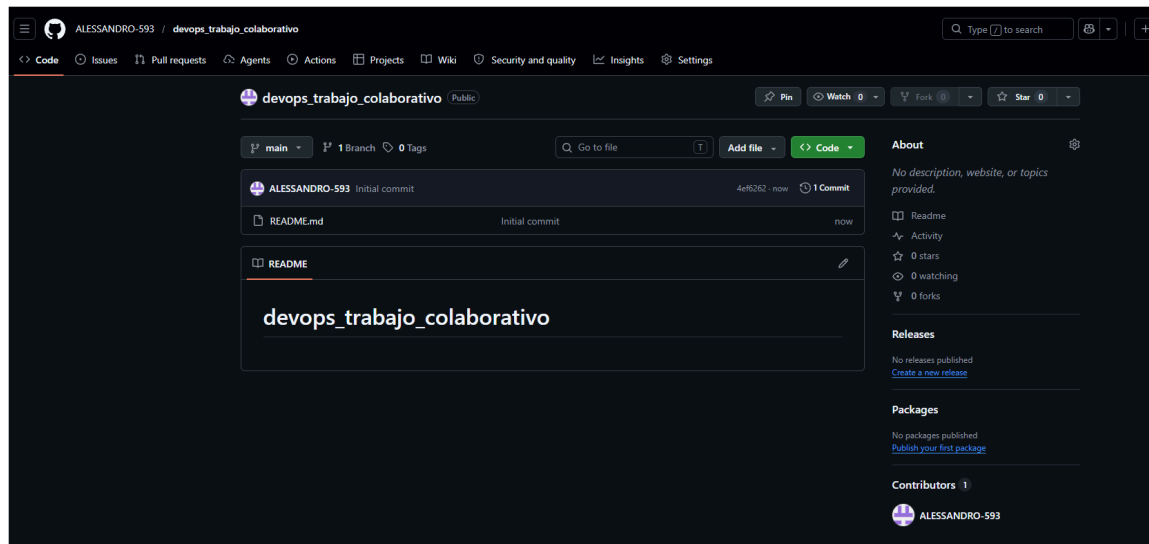
No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

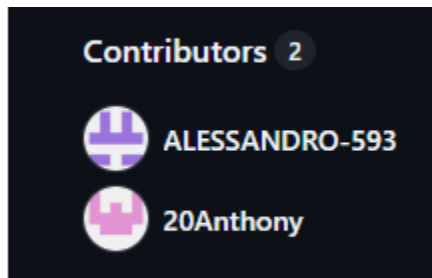
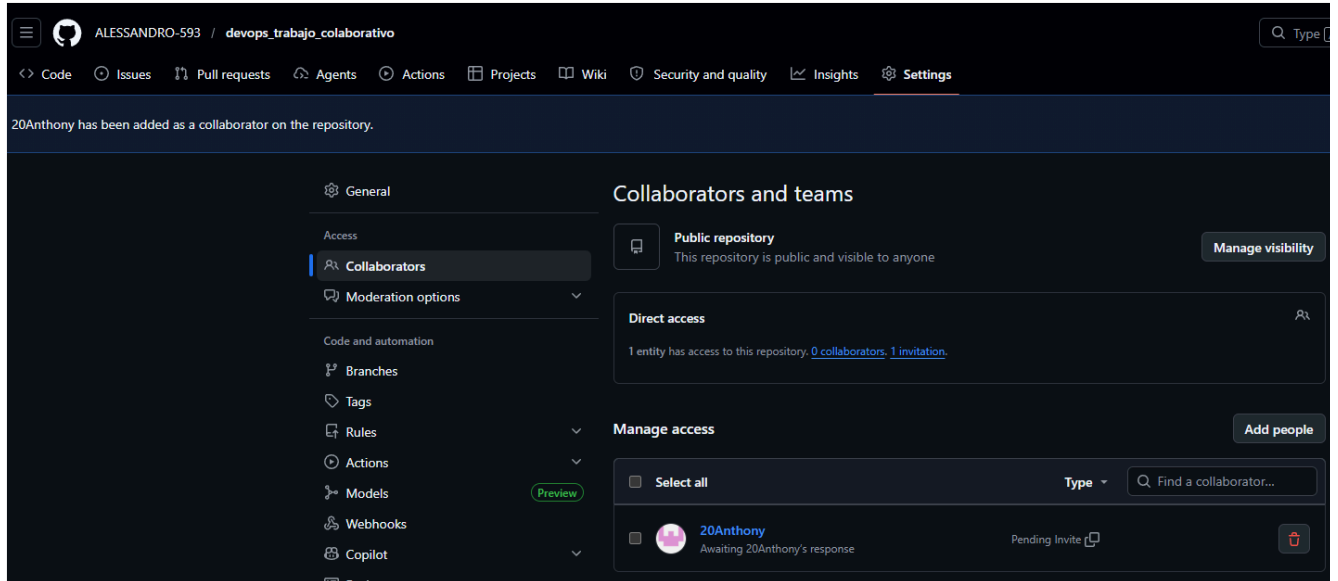
Create repository



Resultado obtenido: Repositorio creado correctamente en la nube, quedando disponible para la vinculación de colaboradores locales y remotos.

2.2 [Erick Basurto] Invitación del Colaborador

Con la finalidad de habilitar el acceso de escritura para Anthony, se ingresó a la sección de administración de accesos del repositorio y se envió una invitación formal de colaboración al usuario de Anthony.



Resultado obtenido

Anthony aceptó la invitación y quedó registrado oficialmente como colaborador del proyecto con permisos completos para interactuar con el repositorio.

2.3 [Erick Basurto] Clonación del Repositorio y commit inicial con proyecto base

Se procedió a clonar el repositorio vacío en la máquina local de Erick mediante el comando `git clone`. Posteriormente, se integró el código fuente del proyecto base (una aplicación web de gestión de tareas estructurada con Flask y PostgreSQL) y se ejecutó un push para subir los archivos a la rama principal (main).

```
PROBLEMAS  SALIDA  TERMINAL  CONSOLA DE DEPURACIÓN  PUERTOS

PS C:\> git clone https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo.git
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
PS C:\> 
```

```

● PS C:\devops_trabajo_colaborativo> git push origin main
info: please complete authentication in your browser...
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 2.88 KiB | 737.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo.git
4ef6262..6e8176f  main -> main
○ PS C:\devops_trabajo_colaborativo> █

```

Repositorio con el proyecto base

The screenshot shows the GitHub interface for the repository 'devops_trabajo_colaborativo' owned by 'ALESSANDRO-593'. The repository is public and has 1 branch (main) and 0 tags. The commit history table shows the following files and their commit details:

File	Commit Message	Commit Hash	Time
.github/workflows	proyecto base	7a36925	2 minutes ago
__pycache__	proyecto base		2 minutes ago
templates	proyecto base		2 minutes ago
README.md	proyecto base		2 minutes ago
app.py	proyecto base		2 minutes ago
requirements.txt	proyecto base		2 minutes ago

Resultado obtenido: El repositorio remoto quedó configurado con la estructura base del proyecto (app.py, requirements.txt, etc.), estableciendo el punto de partida para las ramas independientes.

2.4 [Anthony Alarcón] Clonar y crear rama

Se realizó la clonación del repositorio en el entorno local. Tras sincronizar la rama principal, se utilizó el comando `git checkout -b feature-anthony` para generar un entorno de trabajo aislado orientado al desarrollo de una nueva funcionalidad.


```
MINGW64: c:/Users/HP/Documents/devops_trabajo_colaborativo
HP@DESKTOP-T06VTIV MINGW64 ~
$ cd Documents

HP@DESKTOP-T06VTIV MINGW64 ~/Documents
$ git clone https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo.git
Cloning into 'devops_trabajo_colaborativo'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 15 (delta 0), reused 12 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 8.21 KiB | 841.00 KiB/s, done.

HP@DESKTOP-T06VTIV MINGW64 ~/Documents
$ cd devops_trabajo_colaborativo

HP@DESKTOP-T06VTIV MINGW64 ~/Documents/devops_trabajo_colaborativo (main)
$ code .

HP@DESKTOP-T06VTIV MINGW64 ~/Documents/devops_trabajo_colaborativo (main)
$
```

Comando utilizado:

git checkout -b feature-anthony

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  powershell + v [ ] [ ] ... | [ ] [ ] x
PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git checkout -b feature-anthony
Switched to a new branch 'feature-anthony'
PS C:\Users\HP\Documents\devops_trabajo_colaborativo> |
```

Resultado obtenido: Repositorio clonado de forma local en el equipo y rama de característica feature-anthony activa para la programación sin alterar el código estable.

2.5 [Anthony Alarcon] Realizar commit y envío de la funcionalidad al remoto

Se implementó la suite de pruebas automatizadas con Pytest. Tras verificar el correcto funcionamiento, se prepararon los archivos con **git add .**, se registró el cambio local con **git commit -m "Subir test"** y se publicó la rama a la plataforma de GitHub mediante **git push origin feature-anthony**.

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

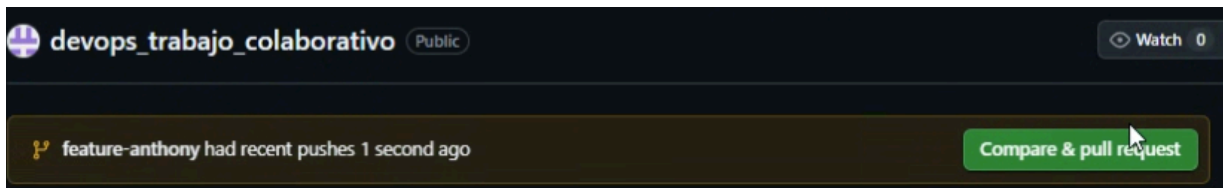
PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git status

nothing added to commit but untracked files present (use "git add" to track)
• PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git add .
• PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git commit -m "Subir test"
[feature-anthony 863fcf0] Subir test
 3 files changed, 173 insertions(+)
  create mode 100644 tests/__pycache__/test_app.cpython-314-pytest-8.2.0.pyc
  create mode 100644 tests/__pycache__/test_app.cpython-314-pytest-9.0.3.pyc
  create mode 100644 tests/test_app.py
• PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git push origin feature-anthony
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 6.16 KiB | 2.05 MiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-anthony' on GitHub by visiting:
remote:   https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo/pull/new/feature-anthony
remote:
To https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo.git
 * [new branch]      feature-anthony -> feature-anthony
• PS C:\Users\HP\Documents\devops_trabajo_colaborativo>
```

Resultado obtenido: Confirmación del registro del commit en el historial local y publicación exitosa de la rama de trabajo en el servidor remoto de GitHub.

2.6 [Anthony Alarcón / Erick Basurto] Apertura, revisión y merge de Pull Request

Anthony inició una solicitud de extracción (Pull Request) en GitHub para integrar feature-anthony a main.



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comp](#)

base: main

compare: feature-anthony

✓ Able to merge. These branches can be automatically merged.



Add a title *

Subir test

Add a description

Write

Preview

H B I                                      

Add your description here...

Markdown is supported

Paste, drop, or click to add files

Create pull request

Pull request de Anthony creado

Filters

is:pr is:open

Labels 9

Milestones 0

New pull request

1 Open

0 Closed

Author

Label

Projects

Milestones

Reviews

Assignee

Sort

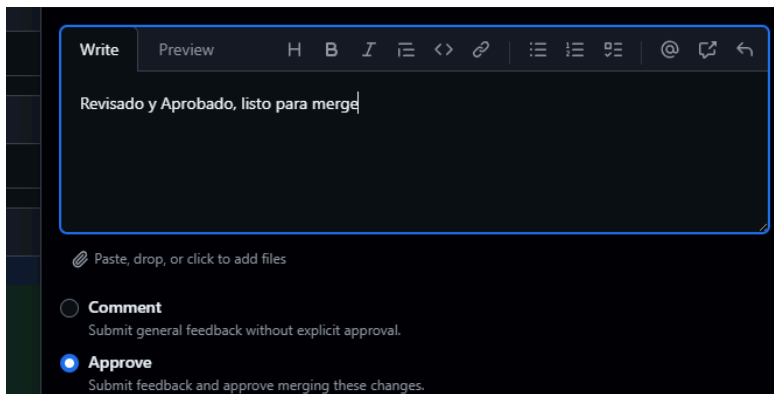
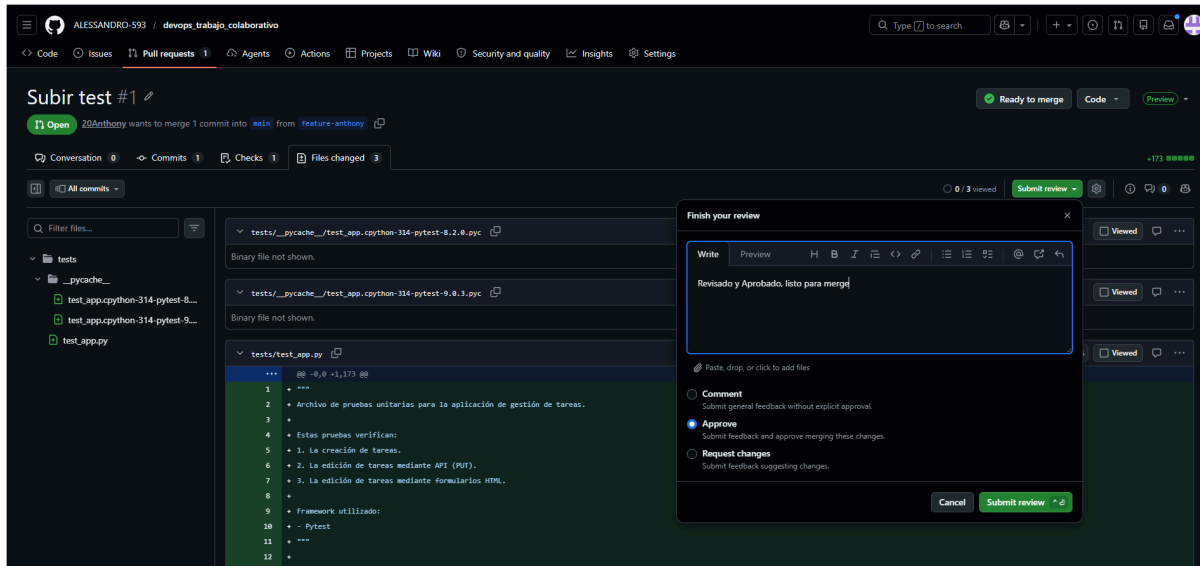
Subir test

#1 opened now by 20Anthony

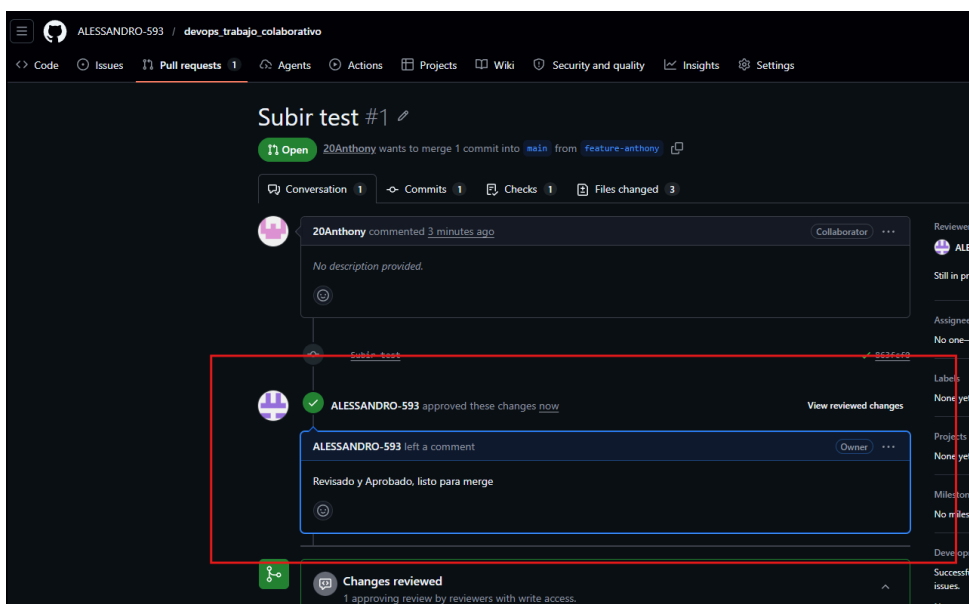
Collaborator

ProTip! Exclude your own issues with `-author:ALESSANDRO-593`.

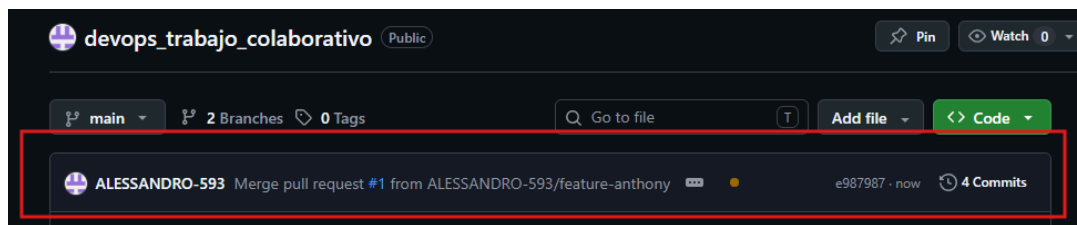
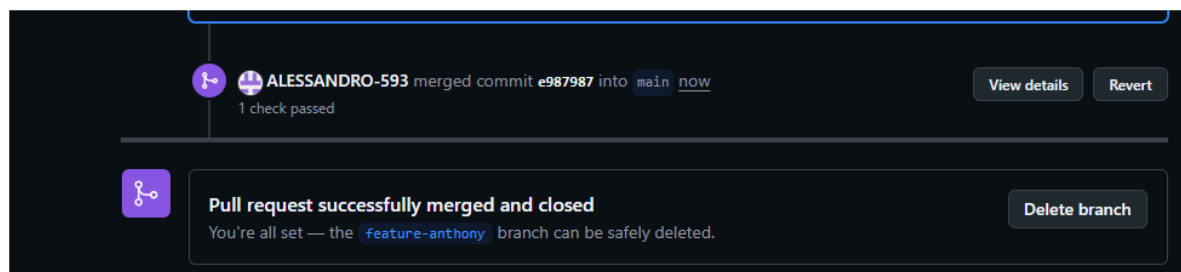
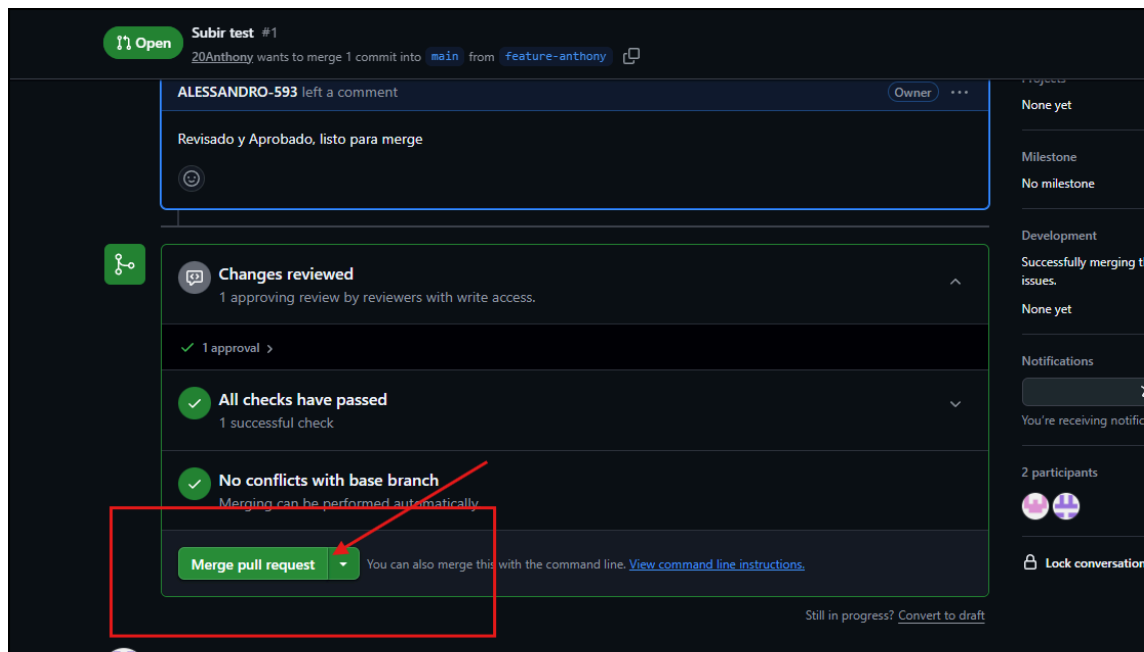
[**Erick Basurto**] se ingresó al panel de discusión del Pull Request y se examinaron los cambios introducidos en el archivo test_app.py y se aprobó la solicitud formalmente añadiendo el comentario de validación correspondiente.



Pull request aprobado



Finalmente se ejecutó la fusión (Merge).



3. INTERCAMBIO DE ROLES Y CONTROL DE CONFLICTOS

3.1 [Erick Basurto] Creación de rama, desarrollo y envío al remoto

Primero se creó localmente la rama `feature-erick` mediante el comando **git checkout -b feature-erick**. Se añadió código para normalizar las entradas de texto del software antes de guardarlas en la base de datos y envió los cambios mediante **git push origin feature-erick**.

Comando utilizado:

git checkout -b feature-erick

```
PS C:\devops_trabajo_colaborativo> git checkout -b feature-erick
Switched to a new branch 'feature-erick'
```

Hacer commit y push en feature-erick

```
PS C:\devops_trabajo_colaborativo> git status
On branch feature-erick
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\devops_trabajo_colaborativo> git add .
PS C:\devops_trabajo_colaborativo> git commit -m "normalizar el nombre de la tarea antes de insertar en la base de datos"
[feature-erick 6c986d1] normalizar el nombre de la tarea antes de insertar en la base de datos
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\devops_trabajo_colaborativo> git push origin feature-erick
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 381 bytes | 381.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'feature-erick' on GitHub by visiting:
remote:   https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo/pull/new/feature-erick
remote:
To https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo.git
 * [new branch]      feature-erick -> feature-erick
PS C:\devops_trabajo_colaborativo>
```

Hacer pull request



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff](#)

 base: **main** < ...

compare: **feature-erick**

✓ **Able to merge.** These branches can be automatically merged.



Add a title *

normalizar el nombre de la tarea antes de insertar en la base de datos

Add a description

Write Preview

H B I           

Add your description here...

 **Markdown is supported**  **Paste, drop, or click to add files**

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Pull request creado

normalizar el nombre de la tarea antes de insertar en la base de datos #2

 **Open** ALESSANDRO-593 wants to merge 1 commit into **main** from **feature-erick**

Conversation 0

Commits 1

Checks 0

Files changed 1



ALESSANDRO-593 commented now Owner

No description provided.



 normalizar el nombre de la tarea antes de insertar en la base de datos

6c906d1



Some checks haven't completed yet
1 in progress check

 Python application / build (pull_request) Started now — This check has started...

 **No conflicts with base branch**
Merging can be performed automatically.

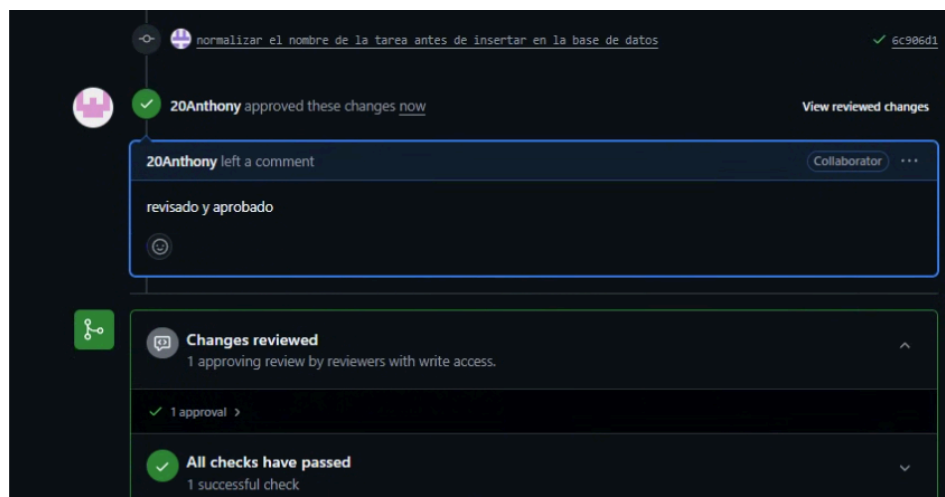
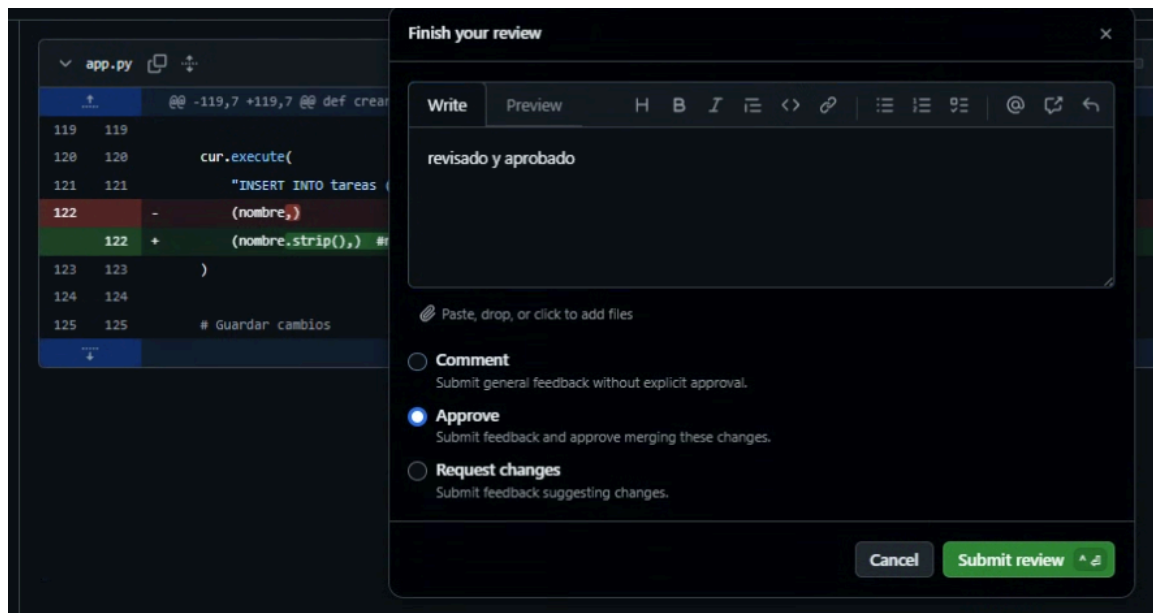
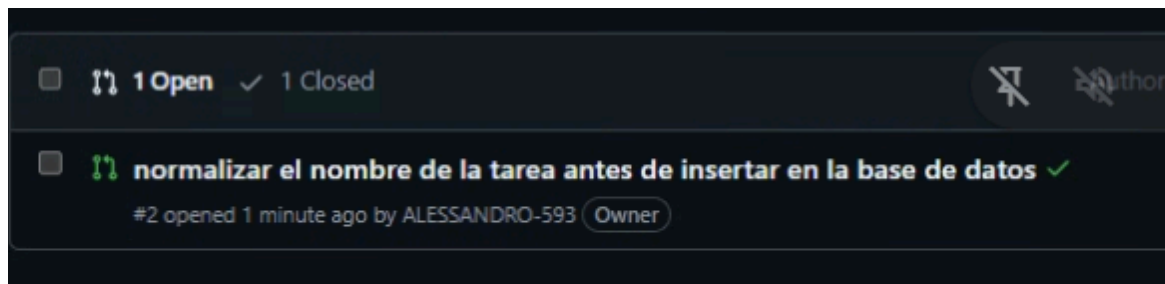
Merge pull request

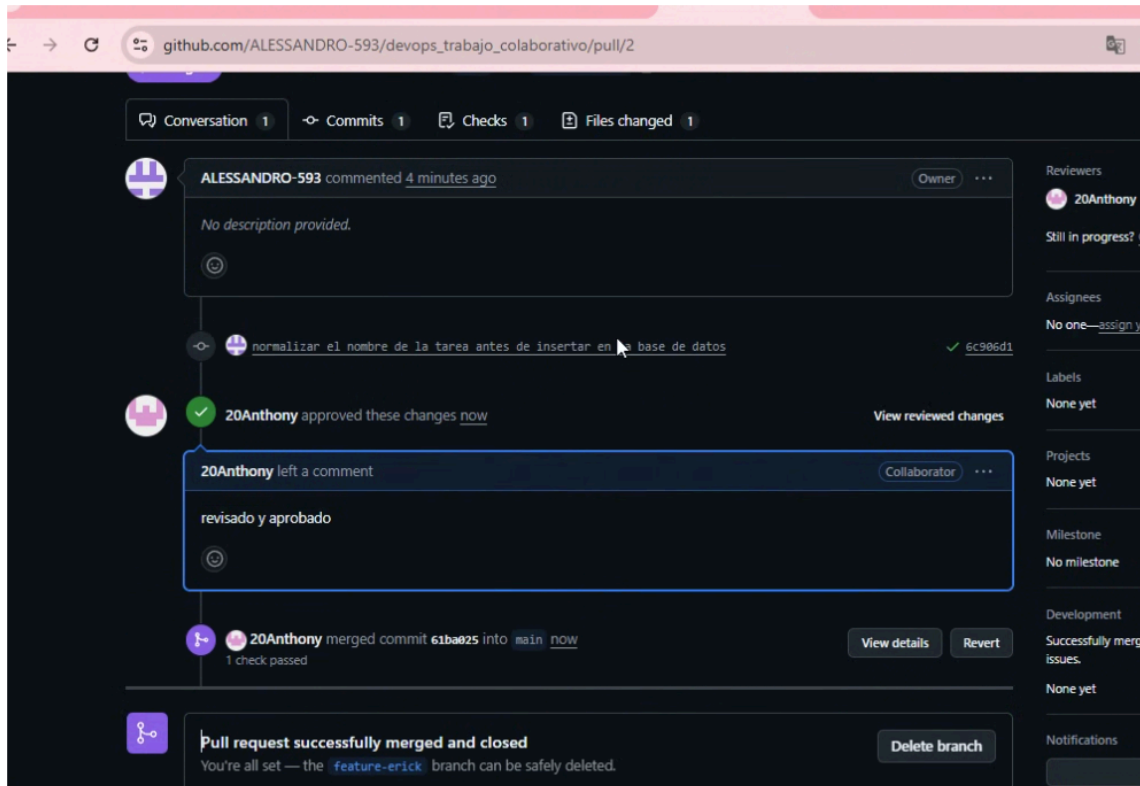
You can also merge this with the command line. [View command line instructions.](#)

Resultado obtenido: Generación de un nuevo Pull Request en el repositorio, quedando a la espera de la revisión de Anthony Alarcón.

3.2 [Anthony Alarcón] Revisión del código de Erick y aprobación de cambios

Se accedió al nuevo Pull Request generado por Erick y se evaluó que la lógica de normalización cumpliera con las directrices requeridas y se procedió a dejar constancia del Code Review aprobando el flujo con la nota "revisado y aprobado".





Resultado obtenido: Flujo aprobado en su totalidad y combinación exitosa del código desarrollado por Erick dentro de la rama estable.

3.3 [Simulación de Conflicto] Gestión y resolución de conflictos en el repositorio

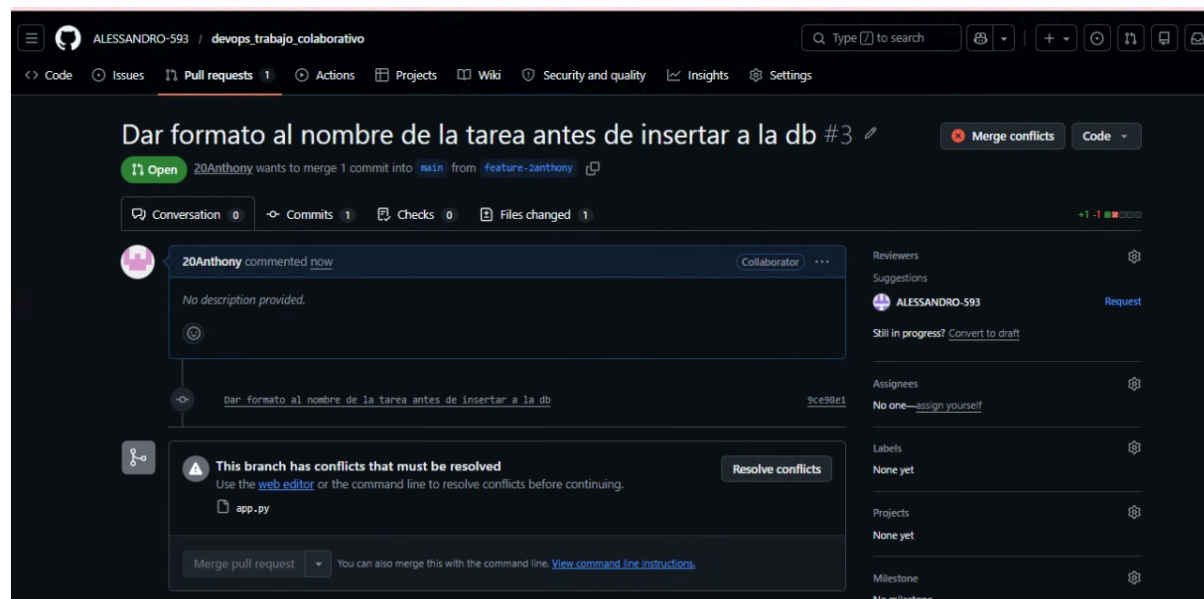
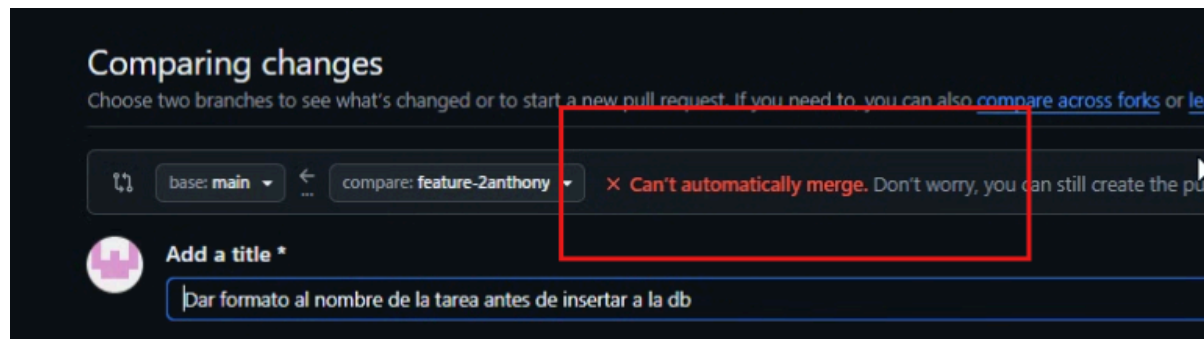
[Anthony] Se editó la misma sección del archivo de persistencia desde una nueva rama remota para dar un formato alternativo, lo que bloqueó la fusión automática debido a un conflicto de líneas concurrentes en Git.

push del cambio realizado por Anthony

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACION  TERMINAL
PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git status

no changes added to commit (use "git add" and/or "git commit -a")
● PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git add .
● PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git commit -m "Dar formato al nombre de la tarea antes de insertar a la db"
[feature-2anthony 9ce98e1] Dar formato al nombre de la tarea antes de insertar a la db
1 file changed, 1 insertion(+), 1 deletion(-)
● PS C:\Users\HP\Documents\devops_trabajo_colaborativo> git push origin feature-2anthony
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 349 bytes | 349.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'feature-2anthony' on GitHub by visiting:
remote:   https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo/pull/new/feature-2anthony
remote:
To https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo.git
 * [new branch]   feature-2anthony -> feature-2anthony
● PS C:\Users\HP\Documents\devops_trabajo_colaborativo>
```

Se realizó un pull request, como la rama creada por Anthony no estaba actualizada, y el cambio se realizó en la misma línea que fue modificada por Erick anteriormente, el Pull Request entró en conflicto.



Se procedió a utilizar el editor web integrado de GitHub para evaluar las diferencias entre ambas ramas, descartar las marcas de conflicto (<<<<<<, =====, >>>>>>) y unificar el método final del script.





The screenshot shows a code editor with a file named `app.py`. The code contains a function that returns a JSON object with an error message. A green checkmark is visible on the left side of the editor. In the top right corner, there is a green button labeled "Commit merge" and a status indicator showing "✓ Resolved". A red rectangle highlights the "Commit merge" button and the "Resolved" status.

Una vez resuelto el conflicto, se procedió a realizar un nuevo commit con los cambios aplicados. Verificando, que cuando volvemos al Pull Request, ya no se presenta el inconveniente de que hay conflictos entre las ramas.



The screenshot shows a GitHub Pull Request interface. The title of the pull request is "Dar formato al nombre de la tarea antes de insertar a la db #3". The pull request is from the `feature-2anthony` branch to the `main` branch. The status bar shows "Conversation 0", "Commits 1", "Checks 0", and "Files changed 1". A comment by "20Anthony" is visible, stating "No description provided." Below the comment, there is a section for checks. The first check is "Some checks haven't completed yet" with "1 in progress check". The second check is "Python application / build (pull_request)" which is "Started now" and "This check has started...". The third check is "No conflicts with base branch" which is "Merging can be performed automatically." and has a green checkmark. At the bottom, there is a "Merge pull request" button and a link to "View command line instructions".

Hacemos el merge



Resultado obtenido: Conflicto resuelto de forma manual, restableciendo el estado óptimo de la rama para permitir la consolidación definitiva del software.

4. AUTOMATIZACIÓN CON GITHUB ACTIONS (INTEGRACIÓN CONTINUA)

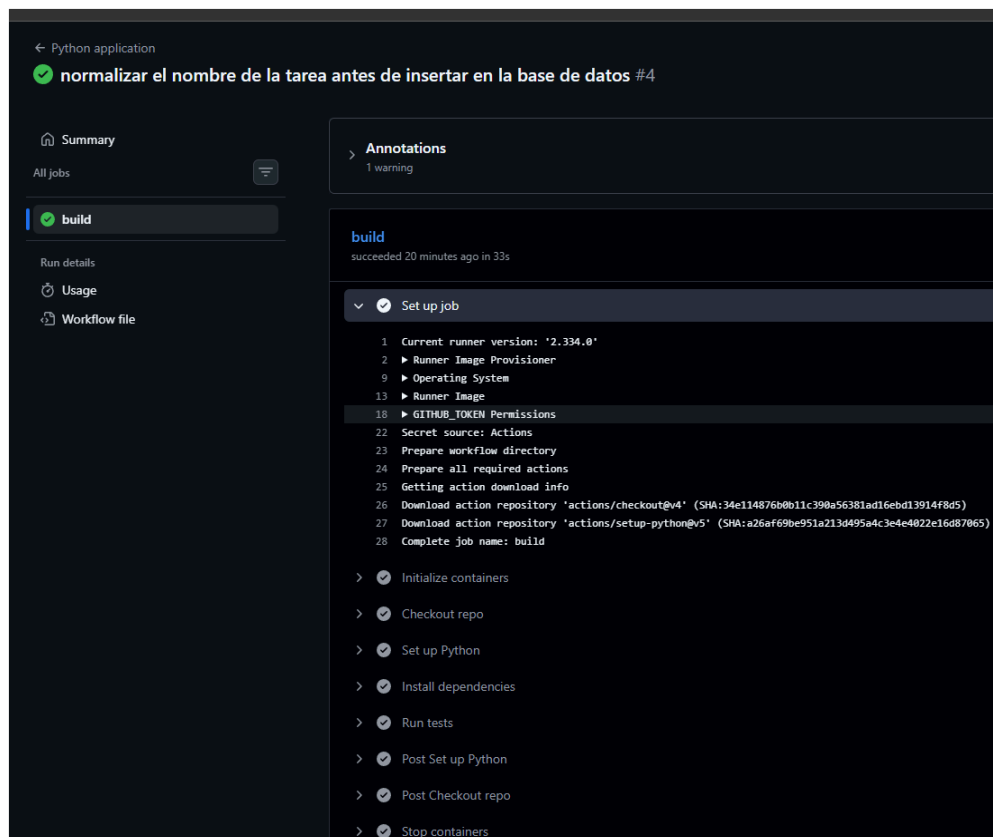
4.1 Configuración y Estado de los Workflows de Integración Continua

Con el fin de implementar prácticas de Integración Continua (CI), se configuró un archivo de flujo de trabajo (workflow) automatizado en GitHub Actions. Este pipeline se encarga de compilar el proyecto, verificar las dependencias y ejecutar las pruebas unitarias automáticamente ante cada evento de push o pull request hacia la rama principal.

ARCHIVO PYTHON-APP.YML

```
README.md  app.py  python-app.yml X
.github > workflows > python-app.yml
1  name: Python application
2
3  on:
4    push:
5      branches: [ "main" ]
6    pull_request:
7      branches: [ "main" ]
8
9  jobs:
10   build:
11     runs-on: ubuntu-latest
12
13     services:
14       postgres:
15         image: postgres:15
16         env:
17           POSTGRES_USER: postgres
18           POSTGRES_PASSWORD: 1234
19           POSTGRES_DB: tareas_db
20         ports:
21           - 5432:5432
22         options: >-
23           --health-cmd="pg_isready -U postgres"
24           --health-interval=10s
25           --health-timeout=5s
26           --health-retries=5
27
28     steps:
29       - name: Checkout repo
30         uses: actions/checkout@v4
31
32       - name: Set up Python
33         uses: actions/setup-python@v5
34         with:
35           python-version: "3.11"
36
37       - name: Install dependencies
38         run: |
39           python -m pip install --upgrade pip
40           pip install -r requirements.txt
41
42       - name: Run tests
43         env:
44           DATABASE_URL: postgres://postgres:1234@localhost:5432/tareas_db
45         run: |
46           pytest
47
```

ESTADOS DE PUSH Y PULL REQUEST LUEGO DE HABER SIDO PROCESADOS POR GITHUB ACTIONS.



Resultado obtenido:

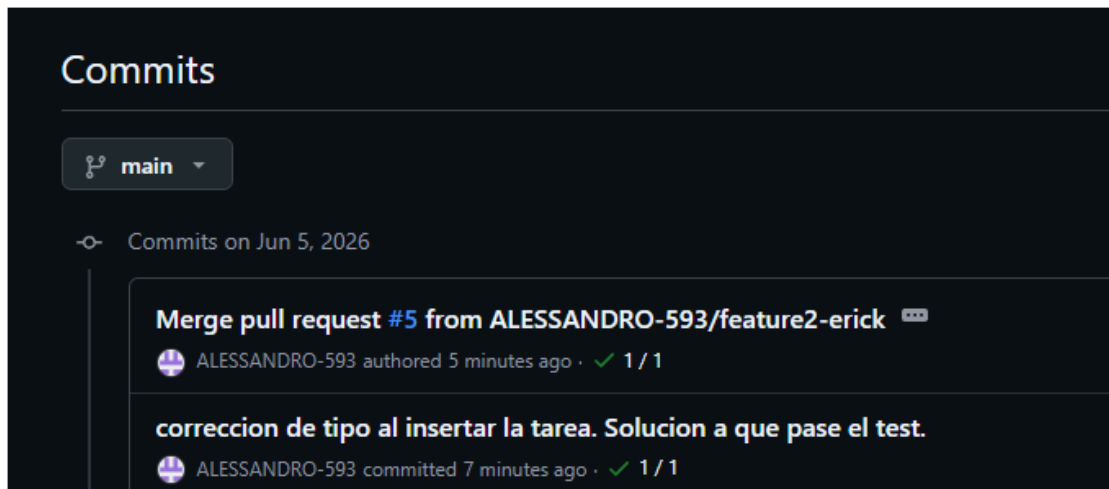
- **Fases iniciales exitosas:** Los primeros commits y despliegues de características de las ramas feature-anthony y feature-erick compilaron y

pasaron los tests automatizados sin inconvenientes, lo que se refleja con los estados en verde dentro del panel de Actions.

- **Fase de conflicto fallida:** El último flujo de trabajo disparado muestra un estado de fallo. Esto se debe a que la concurrencia de cambios introducidos, generó un conflicto de tipado al momento de querer insertar en la base de datos, lo cual generó que los tests fallaran. Como se ve en la imagen.

```
98     ("ViejoNombre",)
99 )
100
101 tarea = cur.fetchone()
102 > tarea_id = tarea[0]
103 E TypeError: 'NoneType' object is not subscriptable
104
105 tests/test_app.py:154: TypeError
106 ===== short test summary info =====
107 FAILED tests/test_app.py::test_crear_tarea - assert 'Original' in '<!DOCTYPE html>\n<html lang="es">\n<head>\n  <!-- Configuración básica del documento -->\n  <meta charset="UTF-8"...</tr>\n\n  <!-- Si no existen tareas -->\n  \n\n  </tbody>\n  </table>\n\n</body>\n</html>'
108 + where '<!DOCTYPE html>\n<html lang="es">\n<head>\n  <!-- Configuración básica del documento -->\n  <meta charset="UTF-8"...</tr>\n\n  <!-- Si no existen tareas -->\n  \n\n  </tbody>\n  </table>\n\n</body>\n</html>' = <bound method Response.get_data of <WrapperTestResponse 4517 bytes [200 OK]>>(as.text=True)
109 +   where <bound method Response.get_data of <WrapperTestResponse 4517 bytes [200 OK]>> = <WrapperTestResponse 4517 bytes [200 OK]>.get_data
110 FAILED tests/test_app.py::test_editar_tarea_api - TypeError: 'NoneType' object is not subscriptable
111 FAILED tests/test_app.py::test_editar_tarea_form - TypeError: 'NoneType' object is not subscriptable
112 ===== 3 failed in 0.17s =====
113 Error: Process completed with exit code 1.
```

- **Fase de corrección:** Una vez que se corrigió el tema del tipado, se hizo un nuevo pull request, y los test pasaron sin inconvenientes, como se muestra en la siguiente captura de pantalla.



4. CONCLUSIONES

[Erick Basurto]

La práctica desarrollada permitió consolidar los conocimientos teóricos sobre flujos de trabajo colaborativos en Git, comprendiendo la importancia de utilizar ramas independientes para desarrollar código sin comprometer la estabilidad del entorno de producción. Asimismo, la simulación de conflictos brindó experiencia crítica para entender los mecanismos con los que Git identifica divergencias de código y la relevancia de la comunicación en equipo; evidenciando de forma práctica que un

conflicto en una sola línea de código es suficiente para detener el flujo de trabajo, romper la arquitectura del software y, en un entorno real, causar fallos en producción que pueden perjudicar la entrega de todo un proyecto.

Repositorio GitHub:

https://github.com/ALESSANDRO-593/devops_trabajo_colaborativo